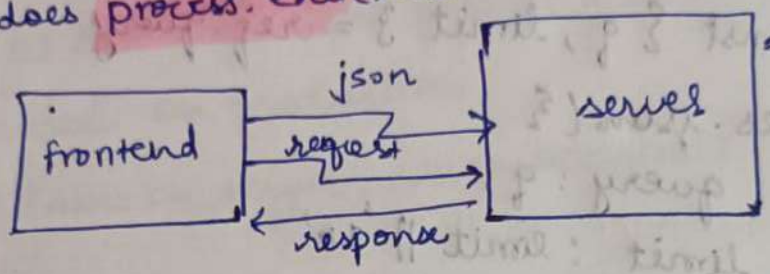
EXPRESS

npm-i-express : to install express

```
fun block_1_basicServer () {
    return new Promise ((resolve) => {
        const app = express()
    }
```

Q What does process.exit() do



⇒ can be written in NodeJS, but not preferred due to complex syntax

To tell ~~prept~~ server, that JSON data will be used :

⇒ app.use (express.json())

/register, /join are called API endpoints and its verbs .get, .post etc

registers handler items at /menu.

```
eg: app.get ('/menu', (req, res) => {
        res.json ({
            items : [ .....
            ]
```

res.json has two work :

1. Set Content Type : 'application/json'
2. Serialize whatever data is sent

Q what is serialization and deserialization.

app.get ('/menu', (_, res) { }
↳ this is a placeholder.

⇒ url like 'chaicode.com/cart?q=biryani&limit=5

Only after you type '?', params are written

So, app.get('/search'), (req, res) ⇒ {
const { q, limit } = req.query
res.json({
query : q,
limit : limit || '10'

⇒ **Route Parameters** ⇒ taken as 'id, also known as path parameter

app.get ('/menu/:id', (req, res) ⇒ {
← handler
const {id} = req.params
res.json({
item : id,
price : 149
})

Ordering of routes is also important.

POST route

app.post ('/order', (req, res) ⇒ {
const{} = req.body
res.status(201).json({
status : 'created',
order
)}
}

res.status is a chainable object.

3

⇒ const server = app.listen (0, async() ⇒ {

const port = server.address().port

const base = ~~http://~~ 127.0.0.1:${port}`; ← localhost or base url

http://

try {

}

catch {

}

}

'0' signals OS that any free port can be assigned. Not recommended on production.

& Research more on server, server.address(). Try console log the server

⇒ Try catch continued

try {

const menuRes = await fetch (`${base}/menu`)

const menuData = await menuRes.json()

console.log ('GET /menu', JSON.stringify(menuData))

const searchResponse = await fetch (`${base}/search?q=biryani&limit=5&page=3`)

Const searchData = searchResponse.json()

const menuItemRes = ~~a~~ wait fetch (`${base}/menu/42`)

Const menuItemData = await menuItemRes.json()

Console.log ('POST /menu', JSON.stringify (menuItemData))

```
const orderRes = await fetch (`${base}/order`, {
    method : 'POST'
    headers : {
        'Content-Type' : 'application/json',
        ~~dish : 'biryani'~~,
    body : JSON.stringify ({
            dish : 'biryani',
            quantity : 2
        })
    })
    const orderData = await orderRes.json()
    console.log ('POST /order', JSON.stringify (orderData))
} catch (error) {
    console.log (error)
}
server.close (() => {
    console.log ("Block 1 served ....")
    resolve()
})
```

This is called kitchen sink, that is a lot of code in one file

⇒ Types of Response

path, handler

1. `app.get ('/text', (req, res) => {`
   `res.send ('Hello from Jaani')`
   `})`

2. `app.get ('/json', (req, res) => {`
   `res.json ({`
   `framework : 'express',`
   `version : '6.1.1'`
   `})`

3. app.get ('/not-found', (req, res) ⇒ {
   res.status(404).json({
      error : "Page Not Found"
   })
})

4. app.get ('/health', (req, res) ⇒ {
   ~~res.status~~ res.sendStatus(200)
})

5. How to redirect ⇒
app.get ('old/menu', (req, res) ⇒ {
//add entry in DB to see how many users are still visiting old route
res.redirect(301, '/new-menu')
↑
special method.

6. Rarely used:
app.get ('/xml', (req, res) ⇒ {
res.type('application/xml').send('<dish>

Q To find more abt res.type

7. To check eg: leetcode runs code fast for premium users, but if endpoint is same, how do we do that. We do it using "custom headers".

app.get ('/custom-headers', (req, res) ⇒ {
res.set('X-powered-By', 'Chai Code');

X-powered-By → name
Chai Code → value

X should be written (is a standard) not mandatory

continued: ⇒

```
res.json({
    message: 'custom headers set'
```

In case of health we send status, in others we are sending body

8. No content, But Endpoint Available
both status and statusCode can be used in this one.

```
app.get('/no-content', (req, res) => {
    res.status(204).end()
})
```

⇒ Routing

```
const routes = {
    1: {
        id: 1,
        name: "Dadar - Andheri Express",
        direction: "North"
    },
    2: {
        id: 2,
        name: "Bandra - Kurla Shuttle",
        direction: "East"
    }
}
```

⇒ This is the database. known as In memory DB

```
let nextId = 3
```

⇒ is done so we can add third value to our db..

```
// list all train
app.get("/routes", (req, res) => {
    res.json(Object.values(routes))
})
```

```
// single route by id
app.get("/routes/:id", (req, res) => {
    ~~req.params~~
const route = routes[req.params.id]
if (!route) return res.status(404).json({error: "No
train on this ~~rou~~ id"});
res.json(route)
})
```

⇒ No validation and zod is being used currently

```
app.post("/routes", (req, res) => {
const newRoute = { id: nextId++, .... req.body}
routes[newRoute.id] = newRoute
res.status(201).json(newRoute)
```

→ you can return anything here status, statusCode etc.

Put cannot be sent without an id

```
⇒ app.put("/routes/:id", (req, res) => {
  const id = req.params.id

// Check if it is an id or not
  if (!routes[id]) return res.status(404).json({error:
    "Something went wrong nhi bhejna hai"});
  routes[id] = { id: Number(id), ..... req.body }

})
```

⇒ **Patch**: Some things are sent which need to be changed, info is in body

```
app.patch("/routes/:id", (req, res) => {
  const id = req.params.id
  if (!routes . . . . . .
  . . . . . }
```

TODO complete this route

⇒ DELETE ⇒ is an operation, no error code is sent

```
app.delete("/routes/:id", (req, res) ⇒ {
    ... ⇒ same as last two
    delete routes['id']
    res.status(204).end()
})
```

Assignment : complete the //TODO part

```
await fetch(`${base}/routes`)
```

⇒ Wildcard ⇒ can take /files/docs/readme.txt

```
app.get("/files/*filepath", (req, res) ⇒ {
    const filepath = req.params.filepath
    res.json({ filepath, type: "wildcard" })
```

The datatype of filepath is array. wildcard means the filepath can have anything eg:

⇒ /files/assets/style.css etc

⇒ ~~app.route~~

⇒ ~~app.~~

⇒ App Route ⇒ a more cleaner syntax

app
- route ("/schedule")
- get ~~((req, res) ⇒ { })~~ (showData)
- post ((req, res) ⇒ { })
- put ((req, res) ⇒ { })
- delete ((req, res) ⇒ { })

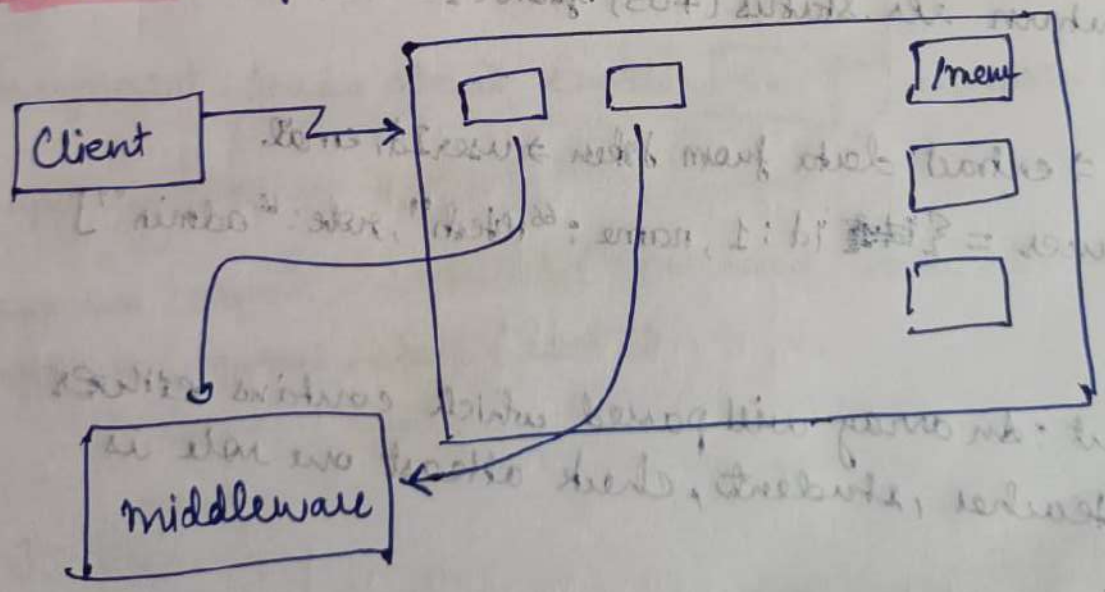
⇒ app.use ⇒ Prefetch Match ⇒ This is run first.

```
app.use("/api", (req, res) ⇒ {
    // its a prefetch match
})
```

The first route that will hit is "/api"

⟹

Controllers is a fun that runs when a route is hit.



// Request Logger : Works like society entry list

```
app.use(req, res, next, error) => { }
    next()
})
```

⇒ Next is a fun that tells fun to pass control to next middleware or route handler in the stack.

⇒ Custom Middleware :→

```
fun authme (req, res, next) {
const token = req.headers['x-auth-token']

//validate
if(!token){
    return res.status(401).json({"error":"No token, pls login"})
}
if (token !== "secret-chaicode") {
    return res.status(403).json({error: "Invalid token"})
}

// token ⇒ extract data from token ⇒ userId, email
req.user = {id: 1, name: "Hitesh", role: "admin"}
```

Assignment : An array will passed which contains either admin, teacher, student, check atleast one role is present.

⇒ Make a rate limiter using factory methods.

```
⇒ fun^n rateLimit (maxRequest) {
    let count = 0
    return (req, res, next) => {
      count ++
      if (count > maxRequest) {
        return res.status(429).json({ error:
                                      " To many requests,
                                        Try after sometime
                                    })
      }
      next()
    }
  }

  const limitedEndpoint = rateLimit(3);
```

To use this middleware ⇒

```
app.get('/limited', limitedEndPoint, (req,res) => {})
```

app.use is in case of built in middleware

Assignment : Study about express.$\boxed{\text{json}}$, express.static

```
app.use(express.json({ limit: '50kb' }))
app.use(express.urlencoded({ extended: true, limit: '50kb' }))
app.use(express.static(root, {
      dotfiles: 'ignore',
      maxAge: 0
  }))
```

Assignment : Design flow of check on excalidraw ⇒

clerk

/Register ⇒ get some data from user
email, name, password
validation
check in db if user alreadyexiess or not
if not create a new user (hash pass)
send email to verify account
send res back, 201

/verify → get token from body
check token validation
if token good, update user object to verify true
else token invalid or expire

/login ⇒ get email and password from body
Validation
if user email is in db or not
if email not found, error
if email is then, check hashed pass
if hash match,
give user a token (cookies or body)
else throw error

Assignment : Make an authentication and e-commerce live above on excalidraw

E-commerce should have ~~to~~ authentication (product, category etc)